Disciplina: Estruturas Lineares	Data: 20/02/2026
Professor(a): Adilson Lima Da Silva	
Alunos(as):	**RGM**
Felipe Costa Monteiro	11231101425

ATIVIDADE SOBRA TAD\

Nos dois exemplos utilizei estruturei o programa em 2 arquivos um sendo principal e outro onde contém a classe com os métodos.

Desafio 1

Main.py(Programa Principal)

```python
# importa a classe Sphere do módulo sphere
from sphere import Sphere


# função principal do programa
def main():
    # comentário: demonstra como usar o TAD Esfera
    try:
        # lê uma string do usuário e tenta converter para float
        r = float(input("Digite o raio da esfera: "))
    except ValueError:
        # se a conversão falhar, avisa e define valor padrão
        print("Raio inválido. Utilizando valor padrão 1.0.")
        r = 1.0

    # cria uma instância de Sphere usando o raio lido
    esfera = Sphere(r)
    # imprime o raio obtido através do método get_radius()
    print(f"Raio: {esfera.get_radius()}")
    # calcula e exibe a área da superfície com 4 casas decimais
    print(f"Área da superfície: {esfera.area():.4f}")
    # calcula e exibe o volume com 4 casas decimais
    print(f"Volume: {esfera.volume():.4f}")


# garante que main() seja executada apenas quando o arquivo for rodado diretamente
if __name__ == "__main__":
    main()
```

UMC UNIVERSIDADE

Sphere.py(Classe)

```python
import math  # módulo de funções matemáticas

class Sphere:
    """Tipo Abstrato de Dados (TAD) representando uma esfera.

    A esfera é definida pelo seu raio. Esta classe fornece
    métodos para obter o raio, a área de superfície e o volume.
    """  # início da definição da classe

    def __init__(self, radius: float):
        """Inicializa a esfera com um raio fornecido.

        Args:
            radius (float): O raio da esfera. Deve ser não-negativo.
        """  # construtor da classe
        if radius < 0:  # validação: não aceitar valores negativos
            raise ValueError("Raio não pode ser negativo")
        self._radius = radius  # armazena o raio na instância

    def get_radius(self) -> float:
        """Retorna o raio da esfera."""  # método acessor simples
        return self._radius

    def area(self) -> float:
        """Calcula e retorna a área de superfície da esfera.

        Fórmula: 4 * pi * r^2
        """  # superfície = 4πr²
        return 4.0 * math.pi * (self._radius ** 2)

    def volume(self) -> float:
        """Calcula e retorna o volume da esfera.

        Fórmula: (4/3) * pi * r^3
        """  # volume = (4/3)πr³
        return (4.0 / 3.0) * math.pi * (self._radius ** 3)

```

UMC UNIVERSIDADE

Desafio 2

Main_conjunto.py(Programa Principal)

```python
# importa a classe IntSet definida em int_set.py
from int_set import IntSet


# função de demonstração principal

def main():
    """Programa principal de demonstração do TAD IntSet."""  # descrição da função
    conjunto = IntSet()  # inicializa conjunto vazio
    # exibe mensagem sobre os valores que serão inseridos
    print("Inserindo alguns valores no conjunto: 4, 1, 7, 1, 3")
    for v in [4, 1, 7, 1, 3]:  # iterar sobre cada valor
        conjunto.insert(v)  # insere no conjunto (duplica ignorada)
    print(f"Conjunto atual: {conjunto}")  # mostra estado atual
    print(f"Menor valor: {conjunto.min_value()}")  # consulta menor elemento
    print(f"Maior valor: {conjunto.max_value()}")  # consulta maior elemento
    print(f"Tamanho: {conjunto.size()}")  # exibe tamanho do conjunto

    print("\nTentando remover 7 e 2...")  # mensagem de remoção
    try:
        conjunto.remove(7)  # tenta remover valor existente
        print("7 removido com sucesso.")
    except ValueError as e:
        print(e)  # mostra erro se não puder remover
    try:
        conjunto.remove(2)  # tenta remover valor ausente
        print("2 removido com sucesso.")
    except ValueError as e:
        print(e)  # exibe mensagem de erro

    print(f"Conjunto final: {conjunto}")  # mostra conjunto após operações


# bloco de proteção para execução como script
if __name__ == "__main__":
    main()  # chama a função principal
```

UMC UNIVERSIDADE

Int_set.py(Classe)

```python
class IntSet:
    """Tipo Abstrato de Dados (TAD) que representa um conjunto de inteiros.

    A implementação usa um vetor (lista) de inteiros internamente e garante
    que não haja elementos duplicados. Disponibiliza operações de inserção,
    remoção, consulta do menor e maior valor e tamanho do conjunto.
    """  # definição da classe IntSet

    def __init__(self, elementos=None):
        """Inicializa o conjunto opcionalmente com uma lista de inteiros.

        Elementos repetidos são ignorados na criação.
        """  # construtor da classe, permite valores iniciais
        self._dados = []  # armazena elementos do conjunto
        if elementos is not None:  # se a lista inicial foi fornecida
            for e in elementos:  # itera pelos elementos
                self.insert(e)  # insere usando método da classe

    def insert(self, value: int) -> None:
        """Insere o inteiro no conjunto se ainda não estiver presente."""  # método de inserção
        if value not in self._dados:  # verifica duplicata
            self._dados.append(value)  # adiciona ao final da lista

    def remove(self, value: int) -> None:
        """Remove o inteiro do conjunto. Levanta ValueError se não existir."""  # método de remoção
        try:
            self._dados.remove(value)  # tenta remover o valor
        except ValueError:
            # se valor não estiver, lança exceção com mensagem clara
            raise ValueError(f"Valor {value} não pertence ao conjunto")

    def min_value(self) -> int:
        """Retorna o menor valor do conjunto. Levanta ValueError se vazio."""  # consulta menor elemento
        if not self._dados:  # se lista estiver vazia
            raise ValueError("Conjunto vazio")
        return min(self._dados)  # usa função built-in min

    def max_value(self) -> int:
        """Retorna o maior valor do conjunto. Levanta ValueError se vazio."""  # consulta maior elemento
        if not self._dados:
            raise ValueError("Conjunto vazio")
        return max(self._dados)  # usa função built-in max

    def size(self) -> int:
        """Retorna o número de elementos do conjunto."""  # retorna comprimento do vetor
        return len(self._dados)

    def __repr__(self) -> str:
        # representação de depuração que mostra o conteúdo da lista
        return f"IntSet({self._dados})"
```